

API Deployment

ORDS - OAuth Configuration Guide

A complete beginner-friendly reference with cURL examples

Info: OAuth 2.0 is an authorization framework that lets applications securely access resources without sharing passwords. A short-lived access token acts as a temporary pass for every API call.

Process Overview

Step	Action	What you create
0	Enable schema	ORDS-enabled schema
1	Global rule config	Server IP / port / path
2	Role + privilege	ERA_USER_ROLE API_DATA_ACCESS
3	OAuth client	TREASURY_APP
4	Set credentials	ERA_TREASURY_APP / ERA_TRS_UFIL_LIVE
5	Get access token	Bearer token (7200 s)
6	Call API	Authenticated JSON response

Warning: Always deploy in a staging environment first. Back up defaults.xml and settings.xml before any changes. Keep the Client Secret confidential at all times.

0

Enable and Register ORDS Schema

Ensure the foundation is in place before writing any SQL

Info: ORDS (Oracle REST Data Services) is the middleware between your Oracle database and the outside world. It translates HTTP requests into SQL and returns JSON responses.

Prerequisites checklist

- ORDS is installed and running on your application server
- The target schema (EMOB) has ORDS REST access enabled
- RestAPI.sql has been successfully deployed
- You have DBA or schema-owner privileges to run the setup SQL

cURL -- Verify ORDS is responding

```
# Quick health check -- should return JSON
curl -s http://172.16.11.3:8080/ufl/efs/ | head -c 200
```

Success: If ORDS responds with JSON, the server is healthy and you can proceed to the next step.

1**Update the Global Rule Configuration**

Tell ORDS where your application server lives

Info: MB_GLOBAL_RULE stores the address of your application server. ORDS uses this to construct callback URLs and correctly route API calls.

SQL -- Update server address

```
UPDATE EMOB.MB_GLOBAL_RULE
SET APP_SERVER_IP    = '172.16.11.3',
    APP_SERVER_PORT  = '8080',
    APP_SERVER_PATH  = '/ufl/efs/';

COMMIT;
```

Parameter	Value	What it means
APP_SERVER_IP	172.16.11.3	Server local network IP address
APP_SERVER_PORT	8080	Port ORDS listens on (HTTP)
APP_SERVER_PATH	/ufl/efs/	Base URL path for eFS REST endpoints

Warning: Use the actual server IP address -- never 'localhost'. Other services need to reach this address over the network.

2**Create ORDS Role and Privilege**

Define who can access which API endpoints

Info: A Role is a security badge category (ERA_USER_ROLE). A Privilege is which API paths that badge unlocks. By setting `la_priv_patterns (1) := '/customers_loan/tax_statement';`

you are instructing ORDS to monitor any incoming request that matches that URL path.

- **The Match:** If a user tries to access [https://your-server.com/ords/schema/customers_loan/tax_statement] (https://your-server.com/ords/schema/customers_loan/tax_statement), ORDS will check if they have the `API_DATA_ACCESS` privilege.
- **Wildcards:** While your example is specific, you can also use the wildcard character (*). For example, `/customers_loan/*` would protect the tax statement endpoint and any other sub-resource under that path.

SQL -- Create role and privilege

```
DECLARE
  la_roles          OWA.vc_arr;
  la_priv_patterns  OWA.vc_arr;
BEGIN
  -- Step A: create the role
  ords.create_role(p_role_name => 'ERA_USER_ROLE');

  -- Step B: link role to URL pattern
  la_roles(1)       := 'ERA_USER_ROLE';
  la_priv_patterns(1) := '/customers_loan/tax_statement';

  -- Step C: define the privilege
  ords.define_privilege(
    p_privilege_name => 'API_DATA_ACCESS',
    p_roles          => la_roles,
    p_patterns       => la_priv_patterns,
    p_label          => 'ERA Data Access',
    p_description    => 'Provides the ability to access eFS Data.'
  );
  COMMIT;
END;
```

SQL -- Verification queries

```
-- Verify role was created
SELECT * FROM user_ords_roles WHERE name = 'ERA_USER_ROLE';

-- Verify privilege was created
```

```
SELECT * FROM user_ords_privileges WHERE name = 'API_DATA_ACCESS';
```

Success: Both queries must return exactly one row. No rows means the DECLARE block failed -- check for ORA errors in your SQL client.

Delete privilege (if you need to recreate)

SQL -- Delete privilege

```
BEGIN
  ords.delete_privilege(p_name => 'API_DATA_ACCESS');
COMMIT;
END;
```

3

Create OAuth Client and Grant the Role

Register your application as an OAuth consumer

Info: An OAuth client is your application's identity. Instead of a human logging in, the app authenticates using a Client ID and Secret using the client_credentials grant type -- designed for server-to-server calls.

SQL -- Create client and grant role

```
BEGIN
  -- Register the client application
  OAuth.create_client(
    p_name          => 'TREASURY_APP',
    p_grant_type     => 'client_credentials',
    p_description    => 'ERA User',
    p_support_email  => 'info@erainfotechbd.com',
    p_privilege_names => NULL    -- linked via role below
  );

  -- Grant the role to this client
  OAuth.grant_client_role(
    p_client_name => 'TREASURY_APP',
    p_role_name   => 'ERA_USER_ROLE'
  );
COMMIT;
END;
```

SQL -- Verify OAuth client

```
-- View all OAuth clients and their auto-generated credentials
SELECT x.* FROM user_ords_clients x;
```

Success: You should see `TREASURY_APP` with a system-generated `CLIENT_ID` and `CLIENT_SECRET`. Note these down -- you will update them in the next step.

4

Set Custom Client ID and Secret

Replace auto-generated credentials with meaningful values

Info: ORDS auto-generates UUID-style credentials. Replacing them with readable values makes Postman environments and config files much easier to manage and audit.

SQL -- Update credentials

```
UPDATE user_ords_clients
SET CLIENT_ID      = 'ERA_TREASURY_APP',
    CLIENT_SECRET = 'ERA_TRS_UFIL_LIVE'
WHERE NAME LIKE 'TREASURY_APP';

COMMIT;
```

Credential	Value
CLIENT_ID	ERA_TREASURY_APP
CLIENT_SECRET	ERA_TRS_UFIL_LIVE
Header prefix	Bearer
Grant type	client_credentials

Warning: Never commit `CLIENT_SECRET` to version control or share it in plain text. Rotate it immediately if it is ever exposed or compromised.

5

Obtain an Access Token

Request a Bearer token using your client credentials

Info: An access token is a temporary string your app sends in every API request header. Think of it as a wristband at an event -- once issued, show it at every door until it expires.

cURL -- Get access token

```
# Request a token using client credentials
curl -s -X GET \
  'http://172.16.11.3:8080/uf1/efs/cbs/get/token' \
  -H 'pClientId: ERA_TREASURY_APP' \
  -H 'pClientSecret: ERA_TRS_UFIL_LIVE'
```

Response -- access token JSON

```
# Expected response (example):
{
  "access_token": "T-PZPjySsQiwKKc9V37PJA",
  "token_type": "bearer",
  "expires_in": 7200
}
```

cURL -- Store token in shell variable

```
# Store token in a shell variable for reuse (Linux / macOS)
TOKEN=$(curl -s -X GET \
  'http://172.16.11.3:8080/uf1/efs/cbs/get/token' \
  -H 'pClientId: ERA_TREASURY_APP' \
  -H 'pClientSecret: ERA_TRS_UFIL_LIVE' \
  | python3 -c "import sys,json;
print(json.load(sys.stdin)['access_token'])")

echo "Token: $TOKEN"
```

Success: The token expires in 7200 seconds (2 hours). After expiry, simply repeat the same curl call to get a fresh token.

6

Call a Protected API Endpoint

Use the Bearer token to access secured data

Info: Every API request to a protected endpoint must include the Authorization header with 'Bearer' followed by the token value. Without it, ORDS returns HTTP 401 Unauthorized.

cURL -- Call protected endpoint (GET)

```
# GET request with the Bearer token
curl -s -X GET \
  'http://172.16.11.3:8080/ufl/efs/customers_loan/tax_statement' \
  -H 'Authorization: Bearer T-PZPjySsQiwKKc9V37PJA' \
  -H 'Content-Type: application/json'
```

cURL -- Using stored shell variable

```
# Using the shell variable from Step 5 (recommended)
curl -s -X GET \
  'http://172.16.11.3:8080/ufl/efs/customers_loan/tax_statement' \
  -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  | python3 -m json.tool # pretty-print the JSON response
```

cURL -- POST request with JSON body

```
# POST example -- submit data to an endpoint
curl -s -X POST \
  'http://172.16.11.3:8080/ufl/efs/customers_loan/apply' \
  -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"customer_id": 12345, "loan_type": "personal"}'
```

Success: HTTP 200 OK with a JSON body means the endpoint is secured and working correctly. HTTP 401 means the token is missing or has expired.

7

Extend Token Lifetime

Increase token expiry from 1 hour to 2 hours

Info: The default token lifetime is 3600 seconds (1 hour). For long-running batch jobs, extending this prevents mid-run authentication failures that would interrupt processing.

Config -- defaults.xml

```
# Open the ORDS configuration file:
# $ORDS_CONF/global/defaults.xml

# Add or update this entry inside the <properties> block:
```

```
<entry key="security.oauth.max.token.lifetime">7200</entry>
```

Shell -- Restart and verify

```
# Restart ORDS (standalone):
java -jar ords.war stop && java -jar ords.war start

# Or restart Tomcat:
systemctl restart tomcat

# Verify the new token lifetime after restart:
curl -s -X GET \
  'http://172.16.11.3:8080/uf1/efs/cbs/get/token' \
  -H 'pClientId: ERA_TREASURY_APP' \
  -H 'pClientSecret: ERA_TRS_UFIL_LIVE' \
  | python3 -c "import sys,json; d=json.load(sys.stdin);
print('expires_in:', d.get('expires_in'))"
```

Success: After restart, new tokens will show expires_in: 7200. Existing tokens retain their original expiry time.

8

Disable SSL Verification (Testing Only)

Prevent certificate errors on internal or dev servers

Danger: NEVER disable SSL verification in production. This setting removes protection against man-in-the-middle attacks and must only be used on isolated test environments.

Info: On test servers using self-signed certificates, ORDS rejects its own callbacks. Disabling SSL verification temporarily unblocks testing without affecting data security on internal networks.

Config -- Disable SSL verification for testing

```
# Older ORDS -- add to $ORDS_CONF/global/defaults.xml:
<entry key="security.verifySSL">false</entry>

# Latest ORDS -- add to /u02/ords_conf/global/settings.xml:
<entry key="security.verifySSL">false</entry>

# Then restart Tomcat:
systemctl restart tomcat
```

cURL -- Skip SSL on client (testing only)


```
# cURL equivalent -- skip SSL on the client side during testing:
curl -s -k -X GET \
  'https://172.16.11.3:8443/ufl/efs/cbs/get/token' \
  -H 'pClientId: ERA_TREASURY_APP' \
  -H 'pClientSecret: ERA_TRS_UFIL_LIVE'

# The -k flag tells cURL to skip certificate verification.
# IMPORTANT: Remove -k in production!
```

9

Test from SQL or TOAD

Verify end-to-end connectivity from inside Oracle

Info: If Postman works but your application does not, testing from within Oracle helps isolate whether the issue is network routing, the database host, or application code.

SQL -- UTL_HTTP request

```
-- Fire an HTTP request from inside Oracle
SELECT UTL_HTTP.REQUEST(
  'http://172.16.11.3:8080/ufl/efs/customers_loan/tax_statement'
) AS RESPONSE
FROM DUAL;
```

Warning: ORA-29273 means the database host cannot reach the ORDS server. Check firewall rules and the UTL_HTTP Access Control List (ACL).

Common errors and fixes

HTTP Code / Error	Meaning	Fix
401 Unauthorized	Token missing/expired	Re-request token (Step 5)
403 Forbidden	Role not granted	Re-run grant_client_role
404 Not Found	Wrong URL path	Check APP_SERVER_PATH
Connection refused	ORDS not running	Start/restart ORDS service
SSL certificate error	Self-signed cert	Add -k flag (testing only)
ORA-29273	DB cannot reach ORDS	Fix firewall / UTL_HTTP ACL

Summary of Key Objects

Object type	Name	Description
ORDS Role	ERA_USER_ROLE	Grants access to API endpoints
Privilege	API_DATA_ACCESS	Protects /customers_loan/tax_statement
OAuth Client	TREASURY_APP	Server app using client_credentials
Client ID	ERA_TREASURY_APP	Human-readable credential identifier
Client Secret	ERA_TRS_UFIL_LIVE	Keep confidential -- never commit to VCS
Token lifetime	7200 sec (2 hours)	Configurable in defaults.xml
Header prefix	Bearer	Required in every Authorization header
SSL Verify	false (testing only)	Disabled via settings.xml on test envs only

Deployment Recommendations

Recommendation	Details
Deploy in staging first	Never apply OAuth changes directly to production.
Backup config files	Save defaults.xml and settings.xml before editing.
Secure the Client Secret	Store in a vault or environment variable, not in code.
Restart after config changes	ORDS won't pick up defaults.xml changes without restart.
Use Postman env variables	Store credentials as {{client_id}} / {{client_secret}}.
Re-enable SSL in production	Remove -k from cURL and re-enable security.verifySSL.
Rotate secrets periodically	Change CLIENT_SECRET every 90 days as best practice.